

Behavioral Interview Prep – HPE Payment Gateway Project

Use the **STAR format** for every answer: **S**ituation → **T**ask → **A**ction → **R**esult

Project Overview (Quick Intro)

Use this when asked: "Tell me about a project you worked on."

"I worked on a **distributed payment processing system** for an HPE academic engineering project. The goal was to build a production-grade payment gateway capable of handling **10,000 requests per second**, using a fully self-hosted Kubernetes stack — replacing managed cloud services like AWS MSK, RDS, and ElastiCache with open-source alternatives (Kafka, PostgreSQL, Redis). I was responsible for **Kubernetes deployment, infrastructure setup, frontend, and monitoring** using Prometheus and Grafana."

★ Experience 1 – Handling Duplicate Payments (Idempotency Problem)

Question type: "Describe a technical challenge and how you solved it."

STAR:

- **S:** During development, we noticed that network retries could cause a payment to be processed multiple times, charging the user twice for the same transaction.
 - **T:** We needed to ensure every payment is processed exactly once, even if the client retries.
 - **A:** We implemented **idempotency keys** — each payment request is assigned a unique key stored in **Redis**. Before processing, the API Gateway checks Redis. If the key already exists, the duplicate request is immediately rejected. If the settlement service restarts mid-transaction, Kafka retains the message so it gets replayed exactly once.
 - **R:** Eliminated duplicate transactions entirely. The system safely handles client retries and service restarts without double-charging users.
-

★ Experience 2 – Service Failures and Dead Letter Queue

Question type: "Tell me about a time you handled a failure gracefully."

STAR:

- **S:** In a distributed system, any microservice (Authorization, Clearing, Settlement) can fail temporarily due to network issues or crashes.
 - **T:** We needed a way to ensure no payment was silently lost when a downstream service was unavailable.
 - **A:** We implemented a **Dead Letter Queue (DLQ)** on Kafka. If a message fails to process, the system retries it **up to 3 times with exponential backoff**. If all retries fail, the message is routed to a Dead Letter Topic for manual investigation and alerting.
 - **R:** No payment transactions were silently dropped. Even during load testing where we simulated service crashes, all transactions were eventually processed or flagged for review.
-

★ Experience 3 – Scaling Under Load (HPA Challenge)

Question type: "Tell me about a time you optimized for performance."

STAR:

- **S:** During JMeter load testing, we saw the API Gateway become the bottleneck when traffic surged – response times spiked and some requests timed out.
 - **T:** We needed the system to handle sudden traffic spikes (simulating Black Friday-scale loads) without manual intervention.
 - **A:** We configured **Kubernetes HPA (Horizontal Pod Autoscaler)**. The API Gateway starts with 20 pods and scales up to 100. When CPU exceeds the threshold (500m average), HPA scales aggressively – up to 100% increase or 10 pods every 15 seconds. We also set a **120-second stabilization window** on scale-down to avoid pod flapping.
 - **R:** During load testing, the API Gateway scaled dynamically to **80 pods**, sustaining close to **10,000 RPS** with stable response times and no cascading failures.
-

★ Experience 4 – Security: WAF and Rate Limiting

Question type: "Tell me about a time you thought about security in a project."

STAR:

- **S:** Since we were building a payment system, security was a top priority. We needed to protect against malicious traffic without impacting legitimate users.
 - **T:** Simulate a DoS-like attack and verify the system remains available for real users.
 - **A:** We deployed **NGINX with ModSecurity WAF** in a public subnet as the edge layer. It blocks SQL injection, XSS, path traversal, and malicious payloads. We also configured NGINX rate limiting to cap requests per client per time window. Backend services remain in a private subnet — completely inaccessible from the public internet.
 - **R:** During DoS simulation, malicious traffic was blocked at the edge. Legitimate payment requests continued processing uninterrupted.
-

★ Experience 5 – Monitoring and Observability

Question type: "How do you ensure your system is healthy in production?"

STAR:

- **S:** With 5+ microservices, 3 database shards, and Kafka brokers running simultaneously, finding bottlenecks manually was impossible.
 - **T:** Build a monitoring and alerting system that gives real-time visibility into system health.
 - **A:** Set up **Prometheus** to scrape metrics (CPU, memory, request count, Kafka consumer lag) from all services. Built **Grafana dashboards** for visualization. Configured smart alerts — e.g., alert if Redis memory > 1.8 GB, if any deployment scales beyond 55 pods, or if a pod enters a crash loop. Also set up the **ELK stack** (Elasticsearch, Logstash, Kibana) for centralized log aggregation across all pods.
 - **R:** When Kafka consumer lag spiked during one test, Grafana alerted us within seconds. We identified the Clearing Service was the bottleneck and scaled it up immediately — avoiding delayed payment processing.
-

★ Experience 6 – Database Sharding Design

Question type: "Tell me about a design decision you made and why."

STAR:

- **S:** We expected high volumes of transactions and a single PostgreSQL instance would become a write bottleneck at 10,000 RPS.
- **T:** Design a database architecture that scales horizontally while keeping financial data consistent.
- **A:** Implemented a **3-shard PostgreSQL cluster** using **consistent hashing on transaction_id** to distribute data evenly. Each shard is independent, so writes are parallelized. Also implemented **range partitioning on created_at** (monthly) on the transactions table, so old data doesn't slow

down queries. Used **double-entry ledger** accounting (debits = credits) to maintain financial integrity.

- **R:** Writes were distributed across 3 nodes, reducing per-shard load by ~67%. Queries on recent transactions remained fast due to partitioning.
-

★ Experience 7 – Avoiding Vendor Lock-in (Architecture Decision)

Question type: "Tell me about a time you made a strategic technical decision."

STAR:

- **S:** The initial approach was to use AWS managed services (MSK for Kafka, ElastiCache for Redis, RDS for PostgreSQL, AWS WAF) – which would tie us completely to AWS.
 - **T:** Design the system so it can be migrated to any cloud or bare metal in the future without rewrites.
 - **A:** Replaced all AWS-managed services with self-hosted open-source alternatives running on Kubernetes – Apache Kafka, Redis, PostgreSQL, and NGINX with ModSecurity. Since Kubernetes is cloud-agnostic, the entire stack can be redeployed on GCP, Azure, or on-premise with minimal changes.
 - **R:** Full portability achieved. Additionally, self-hosting gave us deeper configuration control – for example, NGINX's EWMA load balancing algorithm (not available in AWS ALB) gave better real-time traffic distribution.
-

★ Experience 8 – Cross-Language Microservices (Teamwork)

Question type: "Tell me about a time you collaborated in a team with different skill sets."

STAR:

- **S:** Our team of 5 built services in two different languages – Java (Spring Boot) and C++ – which needed to communicate with each other over Kafka.
- **T:** Ensure that Java and C++ services can produce and consume Kafka messages seamlessly, and that the team stays in sync.
- **A:** We standardized on **Apache Kafka** as the communication layer and agreed on message schemas (JSON) upfront. Each team member owned their service independently: API Gateway (Java), Authorization (C++), Clearing (Java), Settlement (C++), Persistence (Java). We used Kafka topics as clear contracts between services.

- **R:** Despite using different languages, services integrated without issues. Kafka's language-agnostic protocol made cross-team collaboration smooth.

My Role – Quick Summary

For when they ask: "What was YOUR specific contribution?"

"My main responsibilities were:

1. **Overall system architecture** – designed the HLD and LLD, decided the event-driven flow through Kafka topics.
2. **Kubernetes deployment** – wrote all YAML manifests, configured HPA, Ingress, TLS, and namespace isolation.
3. **Monitoring setup** – deployed Prometheus and Grafana, created dashboards, set up alerting rules.
4. **Frontend** – built the React-based dashboard.
5. **Load testing coordination** – ran JMeter tests, analyzed results, and tuned HPA parameters based on findings."

? Common Behavioral Questions & Short Answers

Question	Key Points to Hit
"What was the hardest part?"	Idempotency + DLQ design – ensuring no duplicate or lost transactions in a distributed system
"How did you handle disagreements?"	Team agreed on Kafka JSON schemas upfront; avoided integration fights later
"What would you do differently?"	Add distributed tracing (Jaeger/Zipkin) early; debugging cross-service issues without it was hard
"What did you learn?"	Distributed systems are hard – failure is the normal state. Design for failure from day one
"How did you meet a deadline?"	Divided services by ownership; each person independently built and tested their service
"What are you most proud of?"	Scaling to 10,000 RPS on a self-hosted stack – no managed services, pure engineering

[!TIP]

Always end your STAR answers with a **quantified result** if possible (e.g., "10,000 RPS", "80 pods", "3 retries", "3 DB shards"). Numbers make answers memorable and credible.

